Test Time Compute Backup Speaking Script

Additional explanations for questions and discussion

Slide 21 Constructing Revision Training Data

The revision model needs examples of how earlier answers can lead to a corrected answer. The authors independently sample sixty-four full solutions to a training question at a higher temperature.

For each correct target, they select zero to four incorrect answers from the same set as context. The number is sampled uniformly from those five options. Character edit distance helps prioritize incorrect answers related to the correct one.

The small equation example shows the purpose. Both solutions reach two x equals four. The incorrect reference then says x equals four. The target keeps the valid setup and changes the last calculation to x equals two. The model learns to use earlier work when producing a new solution.

These are training pairs assembled after independent sampling. During inference, the context window simply keeps the most recent four answers in order. Edit distance is used to build the training examples.

Slide 22 Why the Last Revision Can Be Wrong

[Point to the correct middle answer, then the incorrect final revision.]

Training contexts contain incorrect reference answers followed by a correct target. During inference, the earlier answers are generated by the revision model itself, so they can already be correct.

This difference creates a distribution shift. A model may keep revising a solution that was already right. The diagram illustrates a wrong answer becoming correct and then becoming wrong again. It is a schematic example.

Under the authors' naive revision approach, about thirty-eight percent of correct answers were converted to incorrect answers. The denominator is correct answers, rather than every generated answer.

This explains the use of final selection from the history. A verifier or majority voting gives the system a chance to recover a correct answer produced earlier. Selection itself can still make mistakes.

Slide 23 A Complete Beam Search Example

[Walk through one round with six candidates, then repeat the same rule.]

Here is the numerical beam-search procedure used in our earlier overview. Let N be six and M be two. First sample six possible first steps. Score the entire prefix of each with the trained PRM.

Keep the highest-scoring N divided by M prefixes, which is three. From each retained prefix, sample two next steps. We now have six longer candidates again.

Score all six together and repeat. Two descendants of the same prefix can both survive. The method is selecting across the whole pool of candidates. It continues until solutions finish or the maximum of forty expansion rounds is reached.

The final candidates are evaluated using weighted best-of-N. Their matching final answers receive summed scores, and one final answer is returned. N is the size of the candidate pool, while M controls how many continuations each retained prefix receives in this implementation.

Slide 24 Difficulty Estimation and Policy Selection

Difficulty is defined relative to the base model. For each test question, the authors independently sample 2,048 full answers.

Oracle difficulty uses ground-truth checking to estimate the probability that one sampled answer is correct. The questions are ranked by this rate and split into five quantiles, with about one fifth in each group.

Predicted difficulty uses the same sample set but replaces correctness labels with learned verifier scores. The mean final-answer score supplies the ranking. The procedure therefore creates a separate set of five groups. A question can fall in different groups under the two definitions.

For each group and budget, the authors choose the best tested setting and evaluate it using held-out folds. This approximates an ideal per-question policy. The 2,048 samples estimate difficulty and their cost is excluded from the plotted search or revision budgets. This matters for practical efficiency.

Slide 25 Difficulty Bins and Cross Validation

After the questions have been assigned to difficulty groups, the paper uses two-fold cross-validation inside each group.

Imagine one group contains twenty questions. Split them into two sets of ten, A and B. Compare the candidate strategies on A and select the best setting for the budget. Then measure that selected strategy on B.

Next reverse the roles. Choose the setting using B and measure its performance on A. Finally average the two held-out results. The selected setting can differ between the two directions.

The point is to evaluate a strategy on questions different from the ones used to select it. The bin definitions remain based on model-relative difficulty. Oracle and predicted difficulty run this procedure with their respective groupings.

Slide 26 Training Rollouts and Lookahead

The word rollout appears at two different stages. For PRM training, we sample several continuations from a solution prefix to the final answer. Their correctness rate creates a target for learning the value of that prefix.

For lookahead search, the PRM is already trained. At inference time, we temporarily continue a current candidate for up to k extra steps, using temperature zero. We ask the PRM to score the endpoint, then use that value to rank the original current prefix.

The temporary continuation helps decide which prefixes to retain. When k is zero, this becomes ordinary beam search. Larger k uses more computation, so the paper estimates the cost as N multiplied by the quantity k plus one.

The important distinction is the role of the continuation. Training rollouts create supervision. Lookahead uses a trained scorer to choose where to search.

Slide 27 Deriving the Inference Multiplier

Let X and Y be the small model's baseline training and inference FLOPs. Scaling its parameters by M, while keeping token counts fixed, gives a larger-model total cost of M times the sum of X and Y.

The small model's cost with extra inference is X plus aY. Set these totals equal and solve for a. We obtain M plus the product of M minus one and X divided by Y. Using training cost six P D and inference cost two P D gives X divided by Y equal to three divided by R. Substituting this ratio gives the boxed formula. Start with M, then add three times the quantity M minus one divided by R.

The factor two comes mainly from a multiplication and an addition for each weight in the main matrix operations. Training adds backward computation. It computes gradients for weights and for the inputs that pass gradients to earlier layers, giving roughly two more forward-sized matrix calculations. The total is about six P per training token.

P already counts parameters across all layers, so the layer count is included. These are approximate FLOPs estimates. They do not measure wall-clock latency or account exactly for every attention and implementation cost.